STEP — Semester 3

Week 2 — Assignment Problems — Solutions

Week 2 Assignment — 5 Problems

Name: Gursimran Bajwa | **Registration No:** RA2511003011289
Language: Java | **Problems solved:** 5 / 5

Problem 1 — ATM PIN Length Validator

Problem Statement

An ATM app must check that the PIN a customer enters is exactly 4 digits long before allowing them to continue, using only the most basic checks.

Approach

- length() returns the PIN's size and a single if / else decides the message — no loop is needed, as the problem specifies.
- The condition is written as != 4 so the failure case is handled first and the success path is the else branch.
- Three PINs (too short, correct, too long) are tested to show both branches.

Java Program — `AtmPinLengthValidator.java`

```
/**
 * Problem 1 - ATM PIN Length Validator
 * Name: Gursimran Bajwa   |   Reg. No: RA2511003011289
 */
public class AtmPinLengthValidator {

    /* One length() call and a single if / else - no loop needed. */
    public static void checkPinLength(String pin) {
        if (pin.length() != 4) {
            System.out.println("Invalid PIN - must be exactly 4 digits.");
        } else {
            System.out.println("PIN length OK.");
        }
    }

    public static void main(String[] args) {
        String[] pins = { "482", "4820", "12345" };

        for (String pin : pins) {
            System.out.println("PIN entered: \"" + pin + "\"");
            checkPinLength(pin);
            System.out.println();
        }
    }
}
```

Output

```
PIN entered: "482"
Invalid PIN - must be exactly 4 digits.

PIN entered: "4820"
PIN length OK.

PIN entered: "12345"
Invalid PIN - must be exactly 4 digits.
```

Problem 2 — Word Reversal Encoder

Problem Statement

The coding club's "mirror text" mini-game reverses every word in a sentence individually while keeping the word order the same, so "hello club" becomes "olleh bulc".

Approach

- split(" ") separates the sentence into words, preserving their order in the array.
- For each word an inner loop walks from the last character down to index 0, appending into a StringBuilder — building the reverse manually rather than relying only on the library reverse().
- The reversed words are appended to an outer StringBuilder, with a single space added between words but not after the last one.
- StringBuilder is used rather than String concatenation so the work stays O(n) instead of creating a new String on every append.

Java Program — `WordReversalEncoder.java`

```
/**
 * Problem 2 - Word Reversal Encoder ("Mirror Text" Mini-Game)
 * Name: Gursimran Bajwa   |   Reg. No: RA2511003011289
 */
public class WordReversalEncoder {

    /* Reverses every word individually while keeping the word order unchanged. */
    public static String reverseEachWord(String sentence) {
        String[] words = sentence.split(" ");
        StringBuilder result = new StringBuilder();

        for (int i = 0; i < words.length; i++) {
            StringBuilder reversedWord = new StringBuilder();

            for (int j = words[i].length() - 1; j >= 0; j--) {
                reversedWord.append(words[i].charAt(j));
            }

            result.append(reversedWord);
            if (i < words.length - 1) {
                result.append(" ");                // re-join with single spaces
            }
        }
        return result.toString();
    }

    public static void main(String[] args) {
        String[] sentences = { "hello club", "Gursimran Bajwa", "step semester three" };

        for (String sentence : sentences) {
            System.out.println("Input : " + sentence);
            System.out.println("Output: " + reverseEachWord(sentence));
            System.out.println();
        }
    }
}
```

Output

```
Input : hello club
Output: olleh bulc

Input : Gursimran Bajwa
Output: narmisruG awjaB

Input : step semester three
Output: pets retsemes eerht
```

Problem 3 — Product Inventory CSV Parser

Problem Statement

The warehouse team receives inventory updates as CSV lines of the form "ProductName,SKU,Quantity" and needs a parser that splits each line into fields and prints a formatted record.

Approach

- split(",") breaks the line into a String array.
- The array length is validated before any field is read — anything other than exactly 3 fields prints "Invalid Record" and returns early.
- Each field is trim()-ed so stray spaces around commas do not appear in the output.
- The record is printed in the required "Product: ... | SKU: ... | Qty: ..." layout.

Java Program — `ProductInventoryCsvParser.java`

```
/**
 * Problem 3 - Product Inventory CSV Parser
 * Name: Gursimran Bajwa   |   Reg. No: RA2511003011289
 */
public class ProductInventoryCsvParser {

    /* Splits a CSV line into exactly 3 fields and prints a formatted record. */
    public static void parseInventoryRecord(String csvLine) {
        String[] fields = csvLine.split(",");

        if (fields.length != 3) {
            System.out.println("Invalid Record");
            return;
        }

        String productName = fields[0].trim();
        String sku         = fields[1].trim();
        String quantity    = fields[2].trim();

        System.out.println("Product: " + productName + " | SKU: " + sku + " | Qty: " + quantity);
    }

    public static void main(String[] args) {
        String[] lines = {
                "Wireless Mouse,WM-2201,150",
                "Wireless Mouse,150",
                "Mechanical Keyboard,MK-4410,75"
        };

        for (String line : lines) {
            System.out.println("Input: \"" + line + "\"");
            parseInventoryRecord(line);
            System.out.println();
        }
    }
}
```

Output

```
Input: "Wireless Mouse,WM-2201,150"
Product: Wireless Mouse | SKU: WM-2201 | Qty: 150

Input: "Wireless Mouse,150"
Invalid Record

Input: "Mechanical Keyboard,MK-4410,75"
Product: Mechanical Keyboard | SKU: MK-4410 | Qty: 75
```

Problem 4 — Library ISBN Normalizer & Validator

Problem Statement

A library's book-intake scanner must normalise and validate 13-character codes: 3 letters (publisher code) + 4 digits (year) + 6 digits (catalog number). Scanned codes sometimes carry stray spaces or a mixed-case publisher code.

Approach

- normalizeCode() trims the input, then rebuilds it as substring(0,3).toUpperCase() + substring(3) so only the publisher code changes case.
- Validation runs in three ordered stages so the failure message names the exact reason: length must be 13, characters 0–2 must pass Character.isLetter(), and characters 3–12 must pass Character.isDigit(). No regex is used.
- On success, substring() slices out the publisher code, the 4-digit year and the 6-digit catalog number, and a StringBuilder assembles the "[PUB] YEAR: yyyy | CATALOG: nnnnnn" display line.
- Five test codes exercise the valid path and all three distinct failure reasons.

Java Program — `LibraryIsbnValidator.java`

```
/**
 * Problem 4 - Library ISBN Normalizer & Validator
 * Name: Gursimran Bajwa   |   Reg. No: RA2511003011289
 * Valid code = 3 letters (publisher) + 4 digits (year) + 6 digits (catalog) = 13 chars.
 */
public class LibraryIsbnValidator {

    /* Trims stray spaces and uppercases only the 3-character publisher code. */
    public static String normalizeCode(String raw) {
        String trimmed = raw.trim();

        if (trimmed.length() < 3) {
            return trimmed;                        // too short to have a publisher code
        }
        return trimmed.substring(0, 3).toUpperCase() + trimmed.substring(3);
    }

    /* Runs the three validation stages and formats the code when it passes. */
    public static String validateAndFormat(String code) {
        if (code.length() != 13) {
            return "Invalid: code must be exactly 13 characters";
        }

        for (int i = 0; i < 3; i++) {
            if (!Character.isLetter(code.charAt(i))) {
                return "Invalid: publisher code must be 3 letters";
            }
        }

        for (int i = 3; i < 13; i++) {
            if (!Character.isDigit(code.charAt(i))) {
                return "Invalid: year and catalog number must be digits";
            }
        }

        String publisher = code.substring(0, 3);
        String year      = code.substring(3, 7);
        String catalog   = code.substring(7);

        StringBuilder display = new StringBuilder();
        display.append("[").append(publisher).append("] YEAR: ").append(year)
               .append(" | CATALOG: ").append(catalog);

        return display.toString();
    }

    public static void main(String[] args) {
        String[] rawCodes = {
                "  pen2026004251  ",
                "12N2026004251",
                "oxf202600",
```

```
                "mcg20260042X1",
                "  hrp2025000199  "
        };

        for (String raw : rawCodes) {
            String normalized = normalizeCode(raw);
            System.out.println("Raw: \"" + raw + "\"");
            System.out.println("  Normalized: \"" + normalized + "\"");
            System.out.println("  Result: " + validateAndFormat(normalized));
            System.out.println();
        }
    }
}
```

Output

```
Raw: "  pen2026004251  "
  Normalized: "PEN2026004251"
  Result: [PEN] YEAR: 2026 | CATALOG: 004251

Raw: "12N2026004251"
  Normalized: "12N2026004251"
  Result: Invalid: publisher code must be 3 letters

Raw: "oxf202600"
  Normalized: "OXF202600"
  Result: Invalid: code must be exactly 13 characters

Raw: "mcg20260042X1"
  Normalized: "MCG20260042X1"
  Result: Invalid: year and catalog number must be digits

Raw: "  hrp2025000199  "
  Normalized: "HRP2025000199"
  Result: [HRP] YEAR: 2025 | CATALOG: 000199
```

Problem 5 — Stop-Word-Filtered Word Frequency Report

Problem Statement

The T&P team wants word-frequency analysis of feedback paragraphs, with common filler words excluded so the report highlights meaningful themes instead of function words.

Approach

- The text is lowercased and stripped of punctuation with chained replace() calls, so "great," and "great" are counted as the same word.
- split("\\s+") splits on runs of whitespace, which avoids empty entries from double spaces.
- isStopWord() checks each word against the fixed stop-word array and the loop skips any match.
- A HashMap counts the remaining words with getOrDefault(word, 0) + 1, and the entries are copied into a List and sorted by value in descending order before printing.

Java Program — `StopWordFilteredWordFrequency.java`

```
import java.util.HashMap;
import java.util.Map;
import java.util.ArrayList;
import java.util.List;

/**
 * Problem 5 - Stop-Word-Filtered Word Frequency Report
 * Name: Gursimran Bajwa   |   Reg. No: RA2511003011289
 */
public class StopWordFilteredWordFrequency {

    static final String[] STOP_WORDS = { "the", "was", "and", "a", "is", "of", "in" };

    private static boolean isStopWord(String word) {
        for (String stopWord : STOP_WORDS) {
            if (word.equals(stopWord)) return true;
        }
        return false;
    }

    /* Cleans the text, drops stop words, counts the rest, prints by count descending. */
    public static void printFilteredWordFrequency(String feedback) {
        String cleaned = feedback.toLowerCase()
                                 .replace(".", "")
                                 .replace(",", "")
                                 .replace("!", "")
                                 .replace("?", "");

        String[] words = cleaned.trim().split("\\s+");

        Map<String, Integer> frequency = new HashMap<>();
        for (String word : words) {
            if (word.isEmpty() || isStopWord(word)) continue;
            frequency.put(word, frequency.getOrDefault(word, 0) + 1);
        }

        // Sort the unique words by their count, highest first.
        List<Map.Entry<String, Integer>> entries = new ArrayList<>(frequency.entrySet());
        entries.sort((first, second) -> second.getValue() - first.getValue());

        for (Map.Entry<String, Integer> entry : entries) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }

    public static void main(String[] args) {
        String[] feedbacks = {
                "The mentor was great, the session was great and clear.",
                "The training in java was useful and the java labs was practical."
        };

        for (String feedback : feedbacks) {
            System.out.println("Feedback: \"" + feedback + "\"");
            System.out.println("Word frequency report:");
```

```
            printFilteredWordFrequency(feedback);
            System.out.println();
        }
    }
}
```

Output

```
Feedback: "The mentor was great, the session was great and clear."
Word frequency report:
great: 2
mentor: 1
session: 1
clear: 1

Feedback: "The training in java was useful and the java labs was practical."
Word frequency report:
java: 2
practical: 1
labs: 1
training: 1
useful: 1
```